Worksheet 1: Elementary programming

If you are using Julia or Python, we recommend using a jupyter notebook. In WeLearn, you need to submit this file. Please clearly indicate in the markup cells, the number of the question for which you are writing the program. Also, please remember to add documentation through comments in your program.

You may also use scripts and use REPL to evaluate them. In that case, please keep all your files for a particular worksheet in a folder and you may upload the compressed archive of that folder.

Please feel free to ask for help!

1. (2 points) Write a function `is_prime` that checks whether a given integer (n) is prime. For this, check whether any integer from 2 to $[\sqrt{n}]$ divides n. If it does, then the function should return `false` else it should return `true`.

2. (4 points) Write a function that generates all prime numbers between any two given integers.

3. (4 points) Write a function `f(x, N)` that evaluates the series

$$\exp(x) = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

upto N^{th} term, for given x and N. *Note: Please avoid calculating the factorial and express each iterate in terms of the previous one.*

4. (4 points) Write a program that prints the calculated values of `f(x,n)` for $x = 0.1, 0.2, \ldots, 1.0$ for $n = 4, 8, 12, 16, 20$. Your program should print a table where rows correspond to changing x and columns correspond to changing n. The table should be formatted to contain exactly 6 decimal places for each number.

5. (6 points) Write a function that uses the continued fraction

$$\frac{4}{\pi} = 1 + \cfrac{1^2}{2 + \cfrac{3^2}{2 + \cfrac{5^2}{2 + \cfrac{7^2}{2 + \ldots}}}}$$

to evaluate π upto n terms. Also write a function that uses Leibniz formula

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \ldots$$

to evaluate π upto n terms.

Write a program that prints the output of both the functions for $n = 1, 2, \ldots, 10$ (as a formatted table with 8 decimal places for each number). Compare their convergence by plotting them in the same plot versus n.

Worksheet 2: Root finding

If you are using Julia or Python, we recommend using a jupyter notebook. In WeLearn, you need to submit this file. Please clearly indicate in the markup cells, the number of the question for which you are writing the program. Also, please remember to add documentation through comments in your program.

You may also use scripts and use REPL to evaluate them. In that case, please keep all your files for a particular worksheet in a folder and you may upload the compressed archive of that folder.

Please feel free to ask for help!

In all the root-finding methods, one does not look for an *exact* solution. If the function value is within certain small number ϵ from the root, then we stop the search and declare the root is found. The value of ϵ depends on the given problem's physical parameters and the required accuracy. Also, there must be a safe-guarding mechanism of having a maximum allowed iterations (to avoid infinite loops).

Given:

1. Maximum iteration count, `maxiter` = 50
2. ϵ or `eps` = `1.0E-5` (i.e.,10^{-5})
3. Use `numpy`: `import numpy as np`

1. (12 points) Use bisection, Secant and Newton-Raphson to solve the equation $3x = \tan(x)$. Plot the function in the range $(0.0, 1.57)$.
   (a) Define a function to call the method, such as, `bisection(f,L,R,eps,maxiter)`, where `f` is the function whose root is within bracketing interval `L` and `R`.
   (b) For bisection, the function should check if the bracketing interval is propper.
   (c) The function should check if the given input points are roots.
   (d) The function should return the required number of iterations and the root.
   (e) For bisection, starting interval could be [1.2,1.4].
   (f) For Secant the first two points could be [1.2,1.25].
   (g) For Newton-Raphson, the starting point is 1.2.
2. (8 points) Consider a particle moving in an asymmetric one-dimensional double-well potential $V(x) = x^4 + \left(\frac{2x}{3}\right)^3 - x^2$ eV with a total energy -0.125 eV. You need to find the turning points for the particle when it is in either potential well. Plot the function in the range $(-1.25, 1.25)$.
   (a) (4 points) Use Bisection method.
   (b) (4 points) Use Secant method.

Worksheet 3: Numerical integration and differentiation

If you are using Julia or Python, we recommend using a jupyter notebook. In WeLearn, you need to submit this file. Please clearly indicate in the markup cells, the number of the question for which you are writing the program. Also, please remember to add documentation through comments in your program.

You may also use scripts and use REPL to evaluate them. In that case, please keep all your files for a particular worksheet in a folder and you may upload the compressed archive of that folder.

Please feel free to ask for help!

1. (8 points) Numerically estimate $f'(x)$ for $f(x) = sin(x)$ at $x = 2\pi/5$, using
   (a) Forward difference: $f'_n \approx (f_{n+1} - f_n)/h$
   (b) Backward difference: $f'_n \approx (f_n - f_{n-1})/h$
   (c) Central difference: $f'_n \approx (f_{n+1} - f_{n-1})/(2h)$
   (d) Five-point approximation: $f'_n \approx (f_{n-2} - 8f_{n-1} + 8f_{n+1} - f_{n+2})/(12h)$

   for $h = [0.5, 0.2, 0.1, 0.05, 0.02, 0.01, 0.005, 0.002, 0.001, 0.0005, 0.0002, 0.0001]$.
   (a) (3 points) Create a table to record the estimate derivatives using different methods for each h.
   (b) (3 points) Plot the differences between exact values and the estimated values (the error).
   (c) (2 points) Fit the log of the error versus the log of h and compare the efficiencies of the methods.
2. (8 points) Numerically estimate $\int_0^1 f(x)dx$ for $f(x) = e^x$, using the following methods:
   (a) Linear (Trapezoidal): $\int_{x_1}^{x_3} f(x)dx = \frac{h}{2}(f_1 + 2f_2 + f_3) + \mathcal{O}(h^3)$
   (b) Quadratic (Simpson's $\frac{1}{3}$ rule): $\int_{x_1}^{x_3} f(x)dx = \frac{h}{3}(f_1 + 4f_2 + f_3) + \mathcal{O}(h^5)$
   (c) Cubic (Simpson's $\frac{3}{8}$ rule): $\int_{x_1}^{x_4} f(x)dx = \frac{3h}{8}(f_1 + 3f_2 + 3f_3 + f_4) + \mathcal{O}(h^5)$
   (d) Quartic (Boole's rule): $\int_{x_1}^{x_5} f(x)dx = \frac{2h}{45}(7f_1 + 32f_2 + 12f_3 + 32f_4 + 7f_5) + \mathcal{O}(h^7)$
   (a) (4 points) Consider the interval $[0, 1)$. Choose m from $[4, 8, 16, 32, 64]$. For trapezoidal and Simpson's 1/3, $N = 2m + 1$. For Simpson's 3/8, $N = 3m + 1$ and for Boole's method $N = 4m + 1$. Create a table to record the estimate integral using different methods for each $h = (1.0 - 0.0)/(N - 1)$ value.
   (b) (2 points) Plot the differences between exact values and the estimated values (the error).
   (c) (2 points) Fit the log of the error versus the log of h and compare the efficiencies of the methods.
3. (4 points) Use Simpson's 3/8 method to integrate $f(x) = x^{-2/3}(1 - x)^{-1/3}$ from 0 to 1. The exact integral is $2\pi/\sqrt{3}$. Use variable substitutions to handle the singularities.

Worksheet 4: Numerical solutions of ODEs

If you are using Julia or Python, we recommend using a jupyter notebook. In WeLearn, you need to submit this file. Please clearly indicate in the markup cells, the number of the question for which you are writing the program. Also, please remember to add documentation through comments in your program.

You may also use scripts and use REPL to evaluate them. In that case, please keep all your files for a particular worksheet in a folder and you may upload the compressed archive of that folder.

Please feel free to ask for help!

1. (8 points) Consider the equation

$$\frac{dx}{dt} = -xt$$

   (a) (4 points) Use Euler and Midpoint methods to solve the equations using $h = 0.01$ from $t_{\text{ini}} = 0$ to $t_{\text{final}} = 15$. The given initial condition is $x(0) = 1.0$. Plot the solutions along with the exact solution.

   (b) (2 points) Now choose $h = 10.0^n$ for $n = -4$ to $n = -2$ in steps of 0.2. For each h, keep t_{final} fixed and solve the above equations using the two methods. For each h, for each method, estimate the error of the position value of the final point (absolute deviation from the exact solution).

   (c) (2 points) Plot and fit $log_{10}h$ versus log_{10} of the error with a straight line to estimate the errors of each of the method.

2. (12 points) Two coupled first-order equations

$$\frac{dy}{dt} = p$$
$$\frac{dp}{dt} = -4\pi^2 y$$

   define simple harmonic motion with period 1.

   (a) (8 points) Use Euler and Midpoint methods to solve the equations using $h = 0.01$ from $t_{\text{ini}} = 0$ to $t_{\text{final}} = 15$. The given initial condition is $y(0) = 1.0$ and $p(0) = 0.0$. Plot the solutions along with the exact solution.

   (b) (2 points) Now choose $h = 10.0^n$ for $n = -4$ to $n = -2$ in steps of 0.2. For each h, keep t_{final} fixed and solve the above equations using the two methods. For each h, for each method, estimate the error of the position value of the final point (absolute deviation from the exact solution).

   (c) (2 points) Plot and fit $log_{10}h$ versus log_{10} of the error with a straight line to estimate the errors of each of the method.

Worksheet 5: Numerical solutions of ODEs

If you are using Julia or Python, we recommend using a jupyter notebook. In WeLearn, you need to submit this file. Please clearly indicate in the markup cells, the number of the question for which you are writing the program. Also, please remember to add documentation through comments in your program.

You may also use scripts and use REPL to evaluate them. In that case, please keep all your files for a particular worksheet in a folder and you may upload the compressed archive of that folder.

Please feel free to ask for help!

ODE schemes

Euler:
$$y_{n+1} = y_n + f(x_n, y_n)h$$

Midpoint:
$$k1 = hf(x_n, y_n)$$
$$y_{n+1} = y_n + hf(x_n + \frac{h}{2}, y_n + \frac{k1}{2})$$

RK4:
$$k1 = hf(x_n, y_n)$$
$$k2 = hf(x_n + h/2, y_n + k1/2)$$
$$k3 = hf(x_n + h/2, y_n + k2/2)$$
$$k4 = hf(x_n + h, y_n + k3)$$
$$y_{n+1} = y_n + (k1 + 2k2 + 2k3 + k4)/6$$

Verlet:
$$y_1 = y_0 + v_0 h + \frac{1}{2}a_0 h^2$$
$$y_{n+1} = 2y_n - y_{n-1} + a_n h^2$$

Velocity Verlet:
$$y_{n+1} = y_n + v_n h + a_n h^2/2$$
$$v_{n+1} = v_n + h(a_n + a_{n+1})/2$$

Leapfrog:
$$v_{n+1/2} = v_n + ha_n/2$$
$$y_{n+1} = y_n + hv_{n+1/2}$$
$$v_{n+1} = v_{n+1/2} + ha_{n+1}/2$$

---

1. (10 points) Two coupled first-order equations

$$\frac{dy}{dt} = p$$
$$\frac{dp}{dt} = -4\pi^2 y$$

define simple harmonic motion with period 1.

   (a) (6 points) Use Verlet, velocity Verlet, and RK4 methods to solve the equations using $h = 0.01$ from $t_{\text{ini}} = 0$ to $t_{\text{final}} = 15$. The given initial condition is $y(0) = 1.0$ and $p(0) = 0.0$. Plot the solutions along with the exact solution.

   (b) (2 points) Now choose $h = 10.0^n$ for $n = -4$ to $n = -2$ in steps of 0.2. For each h, keep t_{final} fixed and solve the above equations using the two methods. For each h, for each method, estimate the error of the position value of the final point (absolute deviation from the exact solution).

   (c) (2 points) Plot and fit $\log_{10} h$ versus $\log_{10}$ of the error with a straight line to estimate the errors of each of the method.

2. (10 points) A projectile of mass $m = 2$ kg is thrown with a speed of 10 ms^{-1} at an angle 60° *w.r.t.* the horizontal plane. The acceleration due to gravity is 9.8 ms^2. The mass experiences a velocity-dependent drag force with coefficients $\gamma = 2$ kg.s^{-1}.

   (a) (4 points) Construct the equations of motion (in the first order and preferably dimensionless). You may use jupyter notebook to show your equations.

   (b) (4 points) Calculate the trajectory of the mass from the starting point to the point when it hits the ground, by solving the above equations using Euler, velocity Verlet, and RK4 methods.

   (c) (2 points) Modify your code to check the cases of $\gamma = 0$ kg.s^{-1} to $\gamma = 10$ kg.s^{-1}, in steps of 1 kg.s^{-1}. Plot all resulting trajectories in a single graph for comparison.

If you are using a jupyter notebook (recommended), then keep all your programs in a single notebook. A good programming style is to define a function for one task with clearly defined input (arguments) and output. For plots you may use matplotlib (if you are using python) or gnuplot (if you are using c or fortran) or LsqFit module if you are using Julia.

If you are planning to submit separate programs, then please follow the guideline below:

- Keep all files of a worksheet in a single folder.
- Follow a systematic naming convention. You may name the program files as `Q1.py` or `Q1a.py`, `Q1b.py` for question 1 (if you have created multiple files for a single question). The data file should be named as `Q1-data-a.dat` and so on.
- Finally compress the entire folder as a single `.zip` or `.tgz` (using `tar cvfz archive.tgz folder-name/`, and submit the file in WeLearn.

---

1. (20 points) The Van der Pol oscillator is a classic example of a system with non-linear damping. It is defined by the equation

$$\frac{d^2y}{dx} = \mu(1 - y^2)\frac{dy}{dx} - \lambda y.$$

where, μ is the damping coefficient and has a strong influence on the nature of the dynamics. λ is the square of the oscillator freqency. For $\mu = 0$, it is a regular simple harmonic oscillator. For $\mu > 0$, it starts displaying a limit cycle behavior. For $\mu = 5$, it shows periodic slow and sharp changes in y. Naturally, such systems benefit from the use of adaptive time stepping. In this example, you need to solve the above equation for $\mu = 5$, $\lambda = 40$, and from $t = 0.0$ to $t = 20$. The initial condition is $y(0) = 0.5$ and $y'(0) = 0.0$.

   (a) (8 points) Use regular RK4 to solve the equation with small time steps $h = 10^{-4}$.

   (b) (9+1 points) Use the Dormand Prince adaptive method with `abstol =` 10^{-6} and `reltol =` 10^{-8}. How many time steps does it require?

   (c) (2 points) Plot the phase space trajectory of the oscillator (y' versus y).

If you are using a jupyter notebook (recommended), then keep all your programs in a single notebook. A good programming style is to define a function for one task with clearly defined input (arguments) and output. For plots you may use matplotlib (if you are using python) or gnuplot (if you are using c or fortran) or LsqFit module if you are using Julia.

If you are planning to submit separate programs, then please follow the guideline below:

- Keep all files of a worksheet in a single folder.
- Follow a systematic naming convention. You may name the program files as `Q1.py` or `Q1a.py`, `Q1b.py` for question 1 (if you have created multiple files for a single question). The data file should be named as `Q1-data-a.dat` and so on.
- Finally compress the entire folder as a single `.zip` or `.tgz` (using `tar cvfz archive.tgz folder-name/`, and submit the file in WeLearn.

---

1. (20 points) Find the eigenvalues and eigenfunctions for the bound state solutions of Schrödinger equation for the following potentials.

   (a) (10 points)

   $$V(x) = \begin{cases} 0, & \text{for } |x| > 1.0 \\ -V_\circ(1 - x^3)/2, & \text{for} |x| \leq 1 \end{cases}$$

   where, $V_\circ = 40$

   (b) (10 points)

   $$V(x) = x^2.$$

If you are using a jupyter notebook (recommended), then keep all your programs in a single notebook. A good programming style is to define a function for one task with clearly defined input (arguments) and output. For plots you may use matplotlib (if you are using python) or gnuplot (if you are using c or fortran) or LsqFit module if you are using Julia.

If you are planning to submit separate programs, then please follow the guideline below:

- Keep all files of a worksheet in a single folder.
- Follow a systematic naming convention. You may name the program files as `Q1.py` or `Q1a.py`, `Q1b.py` for question 1 (if you have created multiple files for a single question). The data file should be named as `Q1-data-a.dat` and so on.
- Finally compress the entire folder as a single `.zip` or `.tgz` (using `tar cvfz archive.tgz folder-name/`, and submit the file in WeLearn.

---

1. **Cooling a Hot Rod with Fixed-End Temperatures**
   (a) **Physical Setup:**
   A metal rod of length L is held at both ends ($x = 0$ and $x = L$) at a fixed, lower temperature $T_0 = 100$ K. The initial temperature of the rod is uniformly higher, say $T_{\text{init}} = 300\text{K} > T_0$. Let the thermal diffusivity be $\alpha = 10^{-4}$. Thus, you have
   $$u(0,t) = T_0, \quad u(L,t) = T_0, \quad u(x,0) = T_{\text{init}}.$$
   (b) (7 points) **Implementation:**
   Implement a solver that
   - Sets up the tridiagonal system $A\,\mathbf{u}^{n+1} = B\,\mathbf{u}^n$ at each time step.
   - Uses the *Thomas algorithm* for efficient solving.
   - Enforces $u(0,t) = T_0$ and $u(L,t) = T_0$ at every step.

   (c) (3 points) **Plot and Analyze:**
   - Plot the initial temperature distribution ($t = 0$) and the final distribution after some time t_{final}.
   - Observe how the rod cools down to T_0 at both ends.
   - Discuss the long-term solution as $t \to \infty$.

2. **Heating with Different Dirichlet Temperatures at Each End**
   (a) **Physical Setup:**
   Consider the same rod of length L, but now the left end is held at $T_{\text{left}} = 200$ K and the right end at $T_{\text{right}} = 400$ K, where $T_{\text{left}} \neq T_{\text{right}}$. The rod starts from an initial condition $u(x,0) = T_{\text{init}}(x) = 300 + exp(-(x - L/2)^2/2\sigma^2)$ with $\sigma = 0.05$, and the thermal diffusivity is still $\alpha = 10^{-4}$.
   (b) **Dirichlet Boundary Conditions:**
   You have
   $$u(0,t) = T_{\text{left}}, \quad u(L,t) = T_{\text{right}}, \quad u(x,0) = T_{\text{init}}(x).$$
   (c) (7 points) **Numerical Scheme:**
   - Write down the Crank–Nicolson update for interior points $i = 1, \ldots, N-1$.
   - Form the matrices A and B, and describe how the boundary temperatures appear in the right-hand side.
   - Solve repeatedly from t^n to t^{n+1} until some final time t_{final}.

   (d) (3 points) **Results and Steady State:**
   - Plot the temperature profiles at several time steps (e.g. t_1, t_2, ...).
   - Show that eventually the solution approaches a *linear* steady state from T_{left} to T_{right}.
   - Investigate the effect of different Δt, Δx, or α on the convergence speed.

If you are using a jupyter notebook (recommended), then keep all your programs in a single notebook. A good programming style is to define a function for one task with clearly defined input (arguments) and output. For plots you may use matplotlib (if you are using python) or gnuplot (if you are using c or fortran) or LsqFit module if you are using Julia.

If you are planning to submit separate programs, then please follow the guideline below:

- Keep all files of a worksheet in a single folder.
- Follow a systematic naming convention. You may name the program files as `Q1.py` or `Q1a.py`, `Q1b.py` for question 1 (if you have created multiple files for a single question). The data file should be named as `Q1-data-a.dat` and so on.
- Finally compress the entire folder as a single `.zip` or `.tgz` (using `tar cvfz archive.tgz folder-name/`, and submit the file in WeLearn.

---

1. (4 points) Use FFT to calculate the Fourier Transform of the following functions
   (a) (2 points) A square function.
   (b) (2 points) A double slit (difference of two square functions of unequal widths).
2. (16 points) Consider a Gaussian wavepacket $\frac{1}{\sigma\sqrt{2\pi}}\exp(-(x-x_\circ)^2/2\sigma^2)$. It is given that $\sigma = 0.04$ and $x_\circ = -5$. Solve the time dependent Schrödinger equation for $-20 \leq x \leq 20$ for the following potentials.
   (a) (8 points)

$$V(x) = \begin{cases} 0, & \text{for } x < 5.0 \text{ or } x > 7 \\ V_\circ, & \text{for } 5 \leq x \leq 7 \end{cases}$$

   where, $V_\circ = 40$
   (b) (8 points)

$$V(x) = 0.1 \times x^2.$$

If you are using a jupyter notebook (recommended), then keep all your programs in a single notebook. A good programming style is to define a function for one task with clearly defined input (arguments) and output. For plots you may use matplotlib (if you are using python) or gnuplot (if you are using c or fortran) or LsqFit module if you are using Julia.

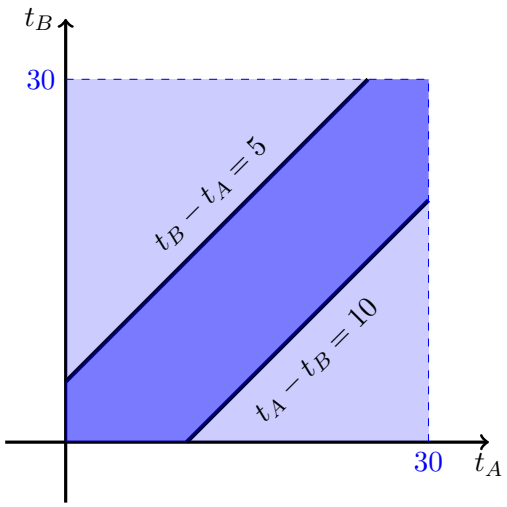
If you are planning to submit separate programs, then please follow the guideline below:

- Keep all files of a worksheet in a single folder.
- Follow a systematic naming convention. You may name the program files as `Q1.py` or `Q1a.py`, `Q1b.py` for question 1 (if you have created multiple files for a single question). The data file should be named as `Q1-data-a.dat` and so on.
- Finally compress the entire folder as a single `.zip` or `.tgz` (using `tar cvfz archive.tgz folder-name/`, and submit the file in WeLearn.

---

1. Write a program that generates a random integer between n_1 and n_2 (as supplied by the user). The function should use the standard uniform random number generator (typically between 0 and 1).

2. Two friends A and B agree to meet in the canteen during their 30 minute tiffin break. A will wait for B for 5 minutes if he shows up first at the canteen. While B, the more patient of the two, will wait for 10 minutes if she were the first one at the canteen. Calculate the probability that the two will meet.



A graphical depiction of the "meeting problem". The light blue rectangle denotes the region in the $t_A - t_B$ plane which covers all possible arrival times – while the dark blue region is where a meeting takes place.

The following lines of code simulate a single such event by generating two random values t_A and t_B between 0 and 30 and will set the variable meet to True if they meet, while if they do not meet, the variable has the value False.

```
from random import random
meet = False
tA = 30*random()
tB = 30*random()
if tA<tB:
    if tB-tA<5:
        meet = True
else:
        if tA-tB<10:
            meet = True
```

By using similar code, try to estimate the probability that the two friends meet. Note that to estimate the probability, we will have to generate many instances of this event and find the fraction of cases where the meeting actually takes place.

3. *Buffon's needle*: consider the floor of a room having parallel strips of wood of width d. The boundary of the strips are marked by thin lines. Now consider a needle of length ℓ thrown on the floor. Using the steps explained in the class, find the probability that the needle crosses a line. Consider both the cases, $\ell \geqslant d$ and $\ell < d$.

4. Consider a random walk process of total number of steps N. Verify that the probability of reaching a position n after N steps is given by

$$P_n = \frac{2}{\sqrt{2\pi N}} \exp\left(-n^2/2N\right)$$

   You may proceed in the following way:

   1. Define a function that simulates taking a walk by using $x_n = x_{n-1} + S_n$, where x_n is the position after n^{th} step and S_n is the n^{th} step (either of $+1$ or -1). The function should take the total steps as argument and return only the final position.
   2. For the walk of N steps, there will be $N + 1$ possible final positions. Create an array of size $N + 1$ to store the occurences of a particular position as discussed in the class.
   3. Perform 100000 walks each having 50 steps. Plot the resulting data along with the exact formula given above. Write your observations as documentation.

5. In this problem, you need to check whether $\langle x^2 \rangle = N$. You may proceed the following way:

   1. Use an array to store the positions of all walkers (say, 100000). All walkers start from the origin (0).
   2. After each step, update the position and calculate $\langle x^2 \rangle$ and store.
   3. Plot $\langle x^2 \rangle$ as a function of steps.

Answer any two questions. Total marks: 50

1. **Dynamics of a Rocket with Drag**: A rocket of structural mass m_0 carries an equal mass m_0 of fuel, giving an initial total mass of $2m_0$. The rocket moves *horizontally* from rest and burns fuel at a constant rate α (kg s^{-1}), ejecting exhaust gas *backward* at speed v_{rel} relative to the rocket. The rocket also experiences a velocity-dependent drag force $F_{drag} = -\gamma v$, where v is its instantaneous velocity. [25]

   Parameters

   Take $m_0 = 1$ kg, $\alpha = 1$ kg s^{-1}, $v_{rel} = 1$ m s^{-1}, and consider three values of the drag coefficient: $\gamma = 0$, $\gamma = \alpha/2$, and $\gamma = \alpha$.

   (a) The fuel is exhausted at time T. What is T in terms of m_0 and α? Write down the instantaneous mass $m(t)$ of the rocket for $0 \le t \le T$ and for $T < t \le 2T$. [2]

   (b) Using the variable-mass form of Newton's second law, show that the equation of motion during Phase 1 ($0 \le t \le T$) is: [3]

   $$(2m_0 - \alpha t)\frac{dv}{dt} = v_{rel}\,\alpha - \gamma v$$

   and write down the equation of motion for Phase 2 ($T < t \le 2T$).

   (c) Solve the Phase 1 equation analytically (with initial condition $v(0) = 0$) to obtain: [5]

   $$v(t) = \frac{v_{rel}\,\alpha}{\gamma}\left[1 - \left(1 - \frac{t}{2T}\right)^{\gamma/\alpha}\right], \qquad \gamma \neq 0$$

   Show that in the limit $\gamma \to 0$, this reduces to the *Tsiolkovsky rocket equation*:

   $$v(t) = v_{rel} \ln\left(\frac{2m_0}{2m_0 - \alpha t}\right)$$

   (d) Write down the Phase 2 solution, expressing $v_1 = v(T)$ (the handoff velocity) explicitly in terms of the given parameters for both $\gamma \neq 0$ and $\gamma = 0$. [3]

   (e) Implement an *RK4 solver* to integrate the equation of motion numerically from $t = 0$ to $t = 2T$, taking care to handle the two phases correctly. Use a step size $dt = 0.001$. [4]

   (f) On a single plot, show $v(t)$ versus t for all three values of γ. For each case, overlay the analytical solution (as a dashed line) on top of the RK4 result (solid line) to verify your numerical solution. Label your axes and include a legend. [3]

   (g) For $\gamma \neq 0$, identify the *terminal velocity* v_{term} that the rocket would asymptotically approach during Phase 1 if it were infinitely long. Express v_{term} in terms of v_{rel}, α, and γ. For the three γ values given, does the rocket approach v_{term} within Phase 1? [2]

   (h) Compute and plot the *position* $x(t)$ from $t = 0$ to $t = 2T$ for all three cases, again overlaying analytical and RK4 results. [3]

2. **Verifying Stokes' law** A classic experiment to verify Stokes' law consists of releasing a small metal sphere from rest at the surface of a viscous liquid and observing its descent under gravity. As the sphere accelerates, the viscous drag increases until the net force vanishes and the sphere falls at a constant *terminal velocity*. In this problem you will analyse this motion both analytically and numerically. [25]

   Background. Consider a sphere of radius r and density ρ_s falling through a quiescent fluid of density ρ_f and dynamic viscosity η. According to *Stokes' law*, a sphere moving at speed v through a viscous fluid at low Reynolds number experiences a drag force

   $$F_{\text{drag}} = 6\pi\eta r v, \tag{1}$$

directed opposite to the motion. Recall also that any object fully submerged in a fluid experiences an upward buoyancy force equal to the weight of fluid displaced (Archimedes' principle). Stokes' law is valid provided the flow remains laminar, i.e. the Reynolds number $\mathrm{Re} = 2\rho_f v r/\eta \ll 1$.

Use the following parameter values throughout:

Quantity	Symbol	Value
Sphere radius	r	1.5×10^{-3} m
Sphere density	ρ_s	7800 kg m^{-3} (steel)
Fluid density	ρ_f	1260 kg m^{-3} (glycerine)
Dynamic viscosity	η	1.49 Pa s (glycerine at 20°C)
Gravitational acceleration	g	9.81 m s^{-2}

(a) **Equation of motion.** By identifying all forces acting on the descending sphere (taking downward as positive), write down its equation of motion. Hence show that it may be written in the form [5]

$$\frac{dv}{dt} = \frac{v_T - v}{\tau}, \tag{2}$$

where τ is a relaxation time and v_T is the terminal velocity. Derive explicit expressions for τ and v_T in terms of the given parameters, and compute their numerical values.

(b) **Analytical solution.**

i. Solve equation (2) with initial condition $v(0) = 0$ to obtain $v(t)$ in closed form. [3]

ii. Show that the time t^* at which the sphere first reaches 99% of the terminal velocity is [3]

$$t^* = \tau \ln 100,$$

and compute its numerical value.

iii. By integrating your expression for $v(t)$, derive a closed-form expression for the depth $x(t)$ reached by the sphere at time t, assuming $x(0) = 0$. Hence find the depth x^* at which $0.99\, v_T$ is first attained, and compute its numerical value. *Hint:* at $t = t^*$ you know $e^{-t^*/\tau}$ exactly. [4]

(c) **Numerical solution.**

i. Implement an **RK4 solver** to integrate equation (2) from $t = 0$ onwards with step size $\Delta t = 10^{-3}$ s and initial condition $v(0) = 0$. Plot $v(t)$ alongside your analytical solution on the same axes and comment on the agreement. [4]

ii. Using your RK4 solution, apply the **bisection method** to find the time t^*_{num} at which the numerically computed velocity satisfies [3]

$$\left| v(t^*_{\mathrm{num}}) - 0.99\, v_T \right| < 10^{-4} \text{ m s}^{-1}.$$

Report t^*_{num} and compare it with your analytical result from part (b)(ii).

iii. Using the **trapezoidal rule** (or Simpson's method), numerically integrate your RK4 velocity profile from $t = 0$ to $t = t^*_{\mathrm{num}}$ to obtain the depth x^*_{num} at which $0.99\, v_T$ is first reached. Compare with your analytical result from part (b)(iii). [3]

3. **Damped oscillation**: A classical particle moves along the x-axis in a potential landscape formed by two Gaussian hills of unequal height and width, creating a valley between them. The particle also experiences a velocity-dependent drag force. As it oscillates back and forth, energy is steadily dissipated and the amplitude of oscillation decreases with each [25]

successive bounce. In this problem you will integrate the equation of motion numerically and locate the turning points on the taller (right) wall.

Background. The potential is

$$V(x) \;=\; V_1 \exp\!\left(-\frac{(x+b)^2}{2\sigma_1^2}\right) \;+\; V_2 \exp\!\left(-\frac{(x-b)^2}{2\sigma_2^2}\right), \tag{3}$$

where $V_1 < V_2$, so that the right hill is taller than the left. The valley floor lies near $x = 0$. The particle of mass m experiences the conservative force $F_{\text{cons}} = -dV/dx$ together with a drag force $F_{\text{drag}} = -\gamma\dot{x}$, giving the equation of motion

$$m\ddot{x} \;=\; -\frac{dV}{dx} - \gamma\dot{x}. \tag{4}$$

Differentiating equation (8), the force due to the potential is

$$-\frac{dV}{dx} \;=\; \frac{V_1(x+b)}{\sigma_1^2}\exp\!\left(-\frac{(x+b)^2}{2\sigma_1^2}\right) \;+\; \frac{V_2(x-b)}{\sigma_2^2}\exp\!\left(-\frac{(x-b)^2}{2\sigma_2^2}\right). \tag{5}$$

Equation (4) is a second-order ODE which may be written as the first-order system

$$\dot{x} = v, \tag{6}$$

$$\dot{v} = \frac{1}{m}\left[-\frac{dV}{dx} - \gamma v\right], \tag{7}$$

to be integrated simultaneously.

Use the following parameter values throughout:

Quantity	Symbol	Value
Particle mass	m	1
Left hill height	V_1	3
Left hill width	σ_1	0.5
Right hill height	V_2	5
Right hill width	σ_2	0.4
Hill separation (half)	b	1
Drag coefficient	γ	0.3
Initial position	x_0	-0.5
Initial velocity	v_0	2

(a) **Potential landscape.**

i. Plot $V(x)$ for $x \in [-3, 3]$. Identify the positions and heights of the two Gaussian hills and the approximate location of the valley floor. Verify that the right hill is taller than the left. [2]

ii. The particle starts at $x_0 = -0.5$ with velocity $v_0 = 2$. Compute $V(x_0)$ and the total initial energy $E_0 = \frac{1}{2}mv_0^2 + V(x_0)$. On your plot of $V(x)$, draw a horizontal line at E_0 and identify the classically accessible region in the absence of drag. Is the particle able to surmount the right hill if there were no drag? [3]

(b) **Numerical integration.**

i. Implement an **RK4 solver** to integrate the system (6)–(7) from $t = 0$ to $t = 30$ with step size $\Delta t = 10^{-3}$ and initial conditions $x(0) = x_0$, $v(0) = v_0$. [3]

ii. Plot $v(t)$ versus t over the full integration range. Describe qualitatively what the particle is doing at each stage: initial descent into the valley, successive bounces off the right and left walls, and eventual settling. Identify by eye the times at which the velocity changes sign. [3]

iii. On a separate figure, plot the **phase portrait** v versus x for the full trajectory. Describe the spiral structure of the trajectory and explain physically why it spirals inward toward a fixed point. What is the fixed point and what does it represent physically? [3]

(c) **Turning points on the right wall.**

A turning point occurs whenever $v(t) = 0$. The turning points on the right wall are those for which $v(t) = 0$ and $x(t) > 0$.

i. By inspecting your plot of $v(t)$, identify approximate time intervals $[t_a, t_b]$ that bracket each of the first three turning points on the right wall (i.e. three consecutive sign changes of v occurring while $x > 0$). [2]

ii. Apply the **bisection method** to each bracketing interval to determine the three turning-point times $t_1^* < t_2^* < t_3^*$ to an accuracy of 10^{-6}. Report the corresponding positions $x(t_1^*)$, $x(t_2^*)$, $x(t_3^*)$. [2]

iii. Compute the total mechanical energy $E(t) = \frac{1}{2}mv(t)^2 + V(x(t))$ at each of the three turning points. Verify that the energy decreases monotonically: $E(t_1^*) > E(t_2^*) > E(t_3^*)$. Explain physically why the turning point positions on the right wall decrease with each successive bounce. [2]

iv. On your plot of $V(x)$, mark the three turning-point positions $x(t_1^*)$, $x(t_2^*)$, $x(t_3^*)$ and draw horizontal lines at the corresponding energies $E(t_1^*)$, $E(t_2^*)$, $E(t_3^*)$. Comment on the pattern. [2]

(d) **Effect of drag.**

i. Repeat the numerical integration for $\gamma = 0$ (no drag) and $\gamma = 0.6$ (stronger drag), keeping all other parameters the same. Plot $v(t)$ for all three values of γ on the same axes. [1]

ii. For $\gamma = 0$, does the particle ever settle? Justify your answer using energy conservation and your phase portrait. [2]

4. **Bound states of a quantum particle** The behaviour of a quantum particle confined to a potential well is governed by the time-independent Schrödinger equation (TISE). In this problem you will find all bound-state energy eigenvalues and the corresponding eigenfunctions for a particle moving in one dimension under a potential of elliptic profile, using the numerical shooting method with slope matching. [25]

Background. Consider a quantum particle of mass m moving along the x-axis in the potential

$$V(x) = \begin{cases} -V_0\sqrt{a^2 - x^2}, & |x| \le a, \\ 0, & |x| > a, \end{cases} \tag{8}$$

where $V_0 > 0$ and $a > 0$. The potential is deepest at the origin with $V(0) = -V_0$, rises smoothly to zero at $x = \pm a$, and vanishes outside $[-a, a]$. Note that the potential has a vertical tangent at $x = \pm a$. The TISE reads

$$-\frac{\hbar^2}{2m}\,\psi''(x) + V(x)\,\psi(x) = E\,\psi(x), \tag{9}$$

where primes denote differentiation with respect to x. For bound states $-V_0 \le E < 0$, and the wavefunction must satisfy $\psi(x) \to 0$ as $|x| \to \infty$.

Work in *natural units* $\hbar = 1$, $2m = 1$, $a = 1$ throughout, so that equation (9) reduces to

$$\psi''(x) = \big[V(x) - E\big]\,\psi(x), \tag{10}$$

with $V_0 = 10$.

(a) **Qualitative analysis.**

i. Plot $V(x)$ for $x \in [-3,\, 3]$. Identify the depth $V(0)$ and the points $x = \pm 1$ where the potential meets zero. Explain physically why bound states exist only for $-V_0 \le E < 0$. [3]

ii. For a bound state with energy $E < 0$, write down the asymptotic form of $\psi(x)$ as $x \to \pm\infty$ in terms of $\kappa = \sqrt{-E}$. Hence justify why $\psi(\pm 3) \approx 0$ is a good approximation for the boundary condition far from the well. [2]

(b) **Shooting method — setup.**

For a trial energy E, integrate equation (10) from both sides toward a common matching point $x_m = +\frac{1}{2}$, using **RK4** with step size $\Delta x = 10^{-3}$.

i. *Left integration.* Seed the left solution at $x = -3$ using the asymptotic decaying form [3]

$$\psi(-3) = e^{-3\kappa}, \qquad \psi'(-3) = \kappa\, e^{-3\kappa},$$

where $\kappa = \sqrt{-E}$, and integrate equation (10) rightward from $x = -3$ to $x = x_m$. Note that the potential has a vertical tangent at $x = -1$; explain why this does not cause a numerical problem for the rightward integration.

ii. *Right integration.* Seed the right solution at $x = +3$ using [2]

$$\psi(+3) = e^{-3\kappa}, \qquad \psi'(+3) = -\kappa\, e^{-3\kappa},$$

and integrate equation (10) leftward from $x = +3$ to $x = x_m$. Explain why the integration must not be started exactly at $x = \pm 1$.

iii. *Matching condition.* Explain why matching the *logarithmic derivative* ψ'/ψ at x_m, rather than ψ and ψ' separately, removes the normalisation ambiguity. Define the mismatch function [3]

$$\mathcal{F}(E) \;=\; \left.\frac{\psi_L'}{\psi_L}\right|_{x_m} \;-\; \left.\frac{\psi_R'}{\psi_R}\right|_{x_m}, \tag{11}$$

where ψ_L and ψ_R denote the left and right solutions respectively, and state the eigenvalue condition.

(c) **Finding the eigenvalues.**

i. Compute and plot $\mathcal{F}(E)$ versus E over the interval $[-V_0,\, 0)$ using a coarse energy grid. Identify by inspection the approximate locations of all zeros of $\mathcal{F}(E)$ and hence the number of bound states supported by the well. [2]

ii. Apply the **bisection method** to each zero identified in part (c)(i) to determine the corresponding eigenvalue E_n to a tolerance of 10^{-6} (in natural units). Tabulate your results. [3]

iii. Verify each eigenvalue by confirming that $|\mathcal{F}(E_n)| < 10^{-6}$. [1]

(d) **Eigenfunctions.**

i. For each eigenvalue E_n, reconstruct the full eigenfunction over $x \in [-3,\, 3]$ by joining ψ_L and ψ_R at x_m: scale ψ_R so that $\psi_L(x_m) = \psi_R(x_m)$, then concatenate the two solutions. [2]

ii. Normalise each eigenfunction numerically using the **trapezoidal rule** so that $\int_{-3}^{3} |\psi_n(x)|^2\, dx = 1$. [1]

iii. Plot all normalised eigenfunctions on a single figure, offset vertically by their respective eigenvalues E_n (i.e. plot $E_n + c\,\psi_n(x)$ for a suitable scale factor c), and superimpose the potential $V(x)$. Count the nodes of each eigenfunction and comment on the pattern. [3]